Objetivos:

I. Introdução ao React Context;

II. Quando usar o React Context?

III. API;

IV. Como funciona?

I. React Context

O React Context fornece uma maneira de passar dados pela árvore de componentes sem precisar passar propriedades manualmente em cada nível.

Em um aplicativo React típico, os dados são passados de cima para baixo (de pai para filho) por meio de propriedades, mas esse uso pode ser trabalhoso para certos tipos de propriedades (por exemplo, preferência de localidade, tema da interface do usuário) que são exigidos por muitos componentes de um aplicativo. O React Context fornece uma maneira de compartilhar valores como esses entre componentes sem a necessidade de passar explicitamente uma propriedade por cada nível da árvore.

O conceito de estado tornou-se complexo à medida que foi necessário compartilhar o estado dentro da aplicação de uma forma global entre os componentes. Para isso, algumas bibliotecas surgiram para resolver essa nova demanda, como o Redux e MobEx. O Redux é a biblioteca mais utilizada para gerenciamento de estados.

O React context API é um gerenciador de estado global e é uma funcionalidade renovada um pouco tempo no ecossistema do React, podendo resolver 90% das soluções do Redux de forma extremamente simples .

II. Quando usar o React Context?

React Context nos permite passar e usar (consumir) dados em qualquer componente que precisamos em nossa aplicação do React sem usar propriedades.

Em outras palavras, o React Context nos permite compartilhar dados (estados) entre nossos componentes de modo mais fácil.

Caso você não saiba o que são estados (states), esse é um termo que se refere a um objeto que armazena dados, que podem mudar ao longo do tempo. É uma maneira de gerenciar e atualizar dados em um componente ou aplicação sem afetar outras partes.

O Context é útil quando você tem valores globais que precisam ser acessados por muitos componentes, como:

- Tema da aplicação (claro/escuro)
- Idioma
- Usuário autenticado
- Carrinho de compras

O Context é usado principalmente quando alguns dados precisam ser acessíveis por vários componentes em diferentes níveis de aninhamento. Aplique-o com moderação, pois dificulta a reutilização de componentes.

Se você deseja apenas evitar passar alguns adereços por vários níveis, a composição de componentes geralmente é uma solução mais simples que o contexto.

III. API

- **React.createContext**

```
const MyContext = React.createContext(defaultValue);
```

Cria um objeto Contexto (context). Quando o React renderiza um componente que assina este objeto Contexto (context), este vai ler o valor atual do Provider superior na árvore que estiver mais próximo.

O argumento defaultValue (valor padrão) é usado apenas quando o componente não corresponder com um Provider acima dele na árvore. Este valor padrão pode ser útil para testar componentes isoladamente, sem envolvê-los. Observação: passando undefined como um valor de Provider não faz com que os componentes consumidores do Provider usem defaultValue.

- **Context.Provider**

```
<MyContext.Provider value={/* algum valor */}>
```

Cada objeto Contexto (context) vem com um componente Provider que permite componentes consumidores a assinarem mudanças no contexto.

O componente Provider aceita uma prop value que pode ser passada para ser consumida por componentes que são descendentes deste Provider. Um Provider pode ser conectado a vários consumidores. Providers podem ser aninhados para substituir valores mais ao fundo da árvore.

Todos consumidores que são descendentes de um Provider serão renderizados novamente sempre que a prop value do Provider for alterada. A propagação do Provider aos seus descendentes (incluído .contextType e useContext), não está condicionada ao método shouldComponenteUpdate, logo, o consumidor é atualizado mesmo quando um componente antepassado ignora uma atualização.

Mudanças são determinadas comparando os valores novos com os anteriores usando o mesmo algorítimo de Object.is.

- **Context.Consumer**

```
<MyContext.Consumer>
  {value => /* renderiza algo baseado no valor do context */}
</MyContext.Consumer>
```

Um componente React que assina mudanças de contexto. Usar este componente permite você assinar a um contexto por um function component.

Requer uma function as a child. A função recebe o valor atual do contexto e retorna um nó React. O argumento value passado para a função será igual ao value da prop do Provider do contexto mais próximo acima na árvore. Se não houver um Provider para este contexto acima, o argumento value será igual a defaultValue que foi passado ao criar o contexto com createContext().

IV. Como funciona?

O Context API tem três partes principais:

1. Criação do Contexto

```
import { createContext } from 'react';
const TemaContext = createContext('claro'); // valor padrão
```

2. Provider (Provedor de Dados)

```
<TemaContext.Provider value="escuro">
  <App />
</TemaContext.Provider>
```

3. Consumer ou useContext (para acessar o valor)

```
import { useContext } from 'react';
const MeuComponente = () => {
  const tema = useContext(TemaContext);
  return <div>Tema atual: {tema}</div>;
};
```

Referências

Documentação oficial do React Context através do link abaixo:

https://legacy.reactjs.org/docs/context.html